Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN:- 22311792

1. Title

Text Feature Extraction using TF-IDF (Term Frequency – Inverse Document Frequency)

---

2. Aim

To implement **TF-IDF** for converting textual data into numerical feature vectors for Natural Language Processing tasks.

---

3. Objectives

- To understand the concept of **Term Frequency (TF)** and **Inverse Document Frequency (IDF)**
- To preprocess text data for NLP applications
- To compute TF-IDF scores for a given set of documents
- To represent text documents in vector form suitable for machine learning models

---

4. Theory

4.1 Natural Language Processing (NLP)

Natural Language Processing is a branch of Artificial Intelligence that enables computers to understand, interpret, and process human language in textual or spoken form.

4.2 Need for Text Vectorization

Machine learning models cannot directly process text. Therefore, text must be converted into numerical form using techniques such as:

- Bag of Words (BoW)

- TF-IDF
- Word Embeddings

4.3 Term Frequency (TF)

Term Frequency measures how frequently a word appears in a document.

TF(t,d)=Number of times term t appears in document dTotal number of terms in document dTF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}TF(t,d)=Total number of terms in document dNumber of times term t appears in document d

---

4.4 Inverse Document Frequency (IDF)

IDF measures how important a word is across the entire document collection.

IDF(t)=log(Ndf(t))IDF(t) = \log\left(\frac{N}{df(t)}\right)IDF(t)=log(df(t)N)

Where:

- NNN = total number of documents
- df(t)df(t)df(t) = number of documents containing term ttt

4.5 TF-IDF

TF-IDF is the product of TF and IDF:

TF-IDF(t,d)=TF(t,d)×IDF(t)TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)TF-IDF(t,d)=TF(t,d)×IDF(t)

It assigns **higher weight to important words** and **lower weight to common words**.

---

5. Program Codes

TF-IDF Algorithm Implementation

```python
import pandas as pd
import numpy as np
```

```python
corpus = [
    'Natural language processing studies human language',
    'Natural language processing analyzes language data',
    'Human language like Hindi is studied in NLP'
]
```

```python
#creating a word set for the corpus
words_set = set()

for doc in  corpus:
    words = doc.split(' ')
    words_set = words_set.union(set(words))

print('Number of words in the corpus:',len(words_set))
print('The words in the corpus: \n', words_set)
```

```
Number of words in the corpus: 14
The words in the corpus:
 {'language', 'data', 'processing', 'analyzes', 'Natural', 'Hindi', 'like', 'NLP', 'is', 'studies', 'Human', 'studied', 'human', 'in'}
```

Computing Term Frequency

```python
#creating a dataframe by the number of documents in the corpus and the word set, and use that information to compute the term frequency (TF)
n_docs = len(corpus)         #·Number of documents in the corpus
n_words_set = len(words_set) #·Number of unique words in the

print(n_docs)
print(n_words_set)

# Convert words_set to a list explicitly before creating the DataFrame.
df_tf = pd.DataFrame(np.zeros((n_docs, n_words_set)), columns=list(words_set))

# Compute Term Frequency (TF)
for i in range(n_docs):
    words = corpus[i].split(' ') # Words in the document
    for w in words:
        df_tf.loc[i, w] = df_tf.loc[i, w] + (1 / len(words)) # Use .loc for indexing

df_tf
```

```
3
14
```

	language	data	processing	analyzes	Natural	Hindi	like	NLP	is	studies	Human	studied	human	in
0	0.333333	0.000000	0.166667	0.000000	0.166667	0.000	0.000	0.000	0.000	0.166667	0.000	0.000	0.166667	0.000
1	0.333333	0.166667	0.166667	0.166667	0.166667	0.000	0.000	0.000	0.000	0.000000	0.000	0.000	0.000000	0.000
2	0.125000	0.000000	0.000000	0.000000	0.000000	0.125	0.125	0.125	0.125	0.000000	0.125	0.125	0.000000	0.125

Computing Inverse Document Frequency

```python
print("IDF of: ")

idf = {}

for w in words_set:
    k = 0    # number of documents in the corpus that contain this word

    for i in range(n_docs):
        if w in corpus[i].split():
            k += 1

    idf[w] =  np.log10(n_docs / k)

    print(f'{w:>15}: {idf[w]:>10}' )
```

```
IDF of:
       language:        0.0
           data: 0.47712125471966244
     processing: 0.17609125905568124
       analyzes: 0.47712125471966244
         Natural: 0.17609125905568124
          Hindi: 0.47712125471966244
           like: 0.47712125471966244
            NLP: 0.47712125471966244
             is: 0.47712125471966244
        studies: 0.47712125471966244
          Human: 0.47712125471966244
        studied: 0.47712125471966244
          human: 0.47712125471966244
             in: 0.47712125471966244
```

Note the IDF value of the word "is"

Computing TF-IDF

```python
df_tf_idf = df_tf.copy()

for w in words_set:
    for i in range(n_docs):
        df_tf_idf[w][i] = df_tf[w][i] * idf[w]

df_tf_idf
```

```
/tmp/ipython-input-2497679995.py:5: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df_tf_idf[w][i] = df_tf[w][i] * idf[w]
```

	language	data	processing	analyzes	Natural	Hindi	like	NLP	is	studies	Human	studied	human	in
0	0.0	0.00000	0.029349	0.00000	0.029349	0.00000	0.00000	0.00000	0.00000	0.07952	0.00000	0.00000	0.07952	0.00000
1	0.0	0.07952	0.029349	0.07952	0.029349	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
2	0.0	0.00000	0.000000	0.00000	0.000000	0.05964	0.05964	0.05964	0.05964	0.00000	0.05964	0.05964	0.00000	0.05964

TF-IDF using sklearn library

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tr_idf_model  = TfidfVectorizer()
tf_idf_vector = tr_idf_model.fit_transform(corpus)
```

```
tf_idf_array = tf_idf_vector.toarray()

print(tf_idf_array)
```

```
[[0.         0.         0.         0.37420726 0.         0.
  0.58121064 0.         0.37420726 0.         0.37420726 0.
  0.49203758]
 [0.46869865 0.46869865 0.         0.         0.         0.
  0.55364194 0.         0.3564574  0.         0.3564574  0.
  0.        ]
 [0.         0.         0.37994462 0.28895767 0.37994462 0.37994462
  0.22440141 0.37994462 0.         0.37994462 0.         0.37994462
  0.        ]]
```

```
words_set = tr_idf_model.get_feature_names_out()

print(words_set)
```

```
['analyzes' 'data' 'hindi' 'human' 'in' 'is' 'language' 'like' 'natural'
 'nlp' 'processing' 'studied' 'studies']
```

```
words_set = tr_idf_model.get_feature_names_out()

print(words_set)
```

```
['analyzes' 'data' 'hindi' 'human' 'in' 'is' 'language' 'like' 'natural'
 'nlp' 'processing' 'studied' 'studies']
```

```
df_tf_idf = pd.DataFrame(tf_idf_array, columns = words_set)

df_tf_idf
```

	analyzes	data	hindi	human	in	is	language	like	natural	nlp	processing	studied	studies
0	0.000000	0.000000	0.000000	0.374207	0.000000	0.000000	0.581211	0.000000	0.374207	0.000000	0.374207	0.000000	0.492038
1	0.468699	0.468699	0.000000	0.000000	0.000000	0.000000	0.553642	0.000000	0.356457	0.000000	0.356457	0.000000	0.000000
2	0.000000	0.000000	0.379945	0.288958	0.379945	0.379945	0.224401	0.379945	0.000000	0.379945	0.000000	0.379945	0.000000

---

6. Result and Outcomes

Result

- The input text documents were successfully converted into TF-IDF feature vectors.
- Each word was assigned a numerical weight based on its importance.
- Common words received lower scores, while unique and meaningful words received higher scores.

Outcomes

- Understood how TF-IDF represents text numerically

- Learned preprocessing and feature extraction for NLP tasks
- Gained practical experience using `TfidfVectorizer` from `scikit-learn`
- Prepared textual data for further machine learning or classification tasks

---

7. Conclusion

TF-IDF is an effective and widely used technique for text feature extraction in Natural Language Processing. It balances term frequency with document importance, making it suitable for tasks such as document classification, clustering, and information retrieval. Through this experiment, we successfully implemented TF-IDF and understood its practical significance in converting unstructured text into structured numerical data.